



**UNIVERSITÀ
DEGLI STUDI
DI UDINE**

HIC SUNT FUTURA

UDINE, 22 SETTEMBRE 2023

PROF. MARIO ROSSI

DIPARTIMENTO POLITECNICO DI
INGEGNERIA E ARCHITETTURA

Strutture dati e algoritmi

Lezione 7 — Ordinamento per fusione



Indice della lezione

- 1** Divide et impera
- 2** Implementazione
- 3** Conclusioni



1 Divide et impera

L'idea generale e il caso dell'ordinamento.



Elenco con rivelazione progressiva

Il paradigma si articola in tre passi.

Elenco con rivelazione progressiva

Il paradigma si articola in tre passi.

- **Divide**: si spezza il problema in sottoproblemi dello stesso tipo.

Elenco con rivelazione progressiva

Il paradigma si articola in tre passi.

- **Divide**: si spezza il problema in sottoproblemi dello stesso tipo.
- **Impera**: si risolvono i sottoproblemi ricorsivamente.

Elenco con rivelazione progressiva

Il paradigma si articola in tre passi.

- **Divide**: si spezza il problema in sottoproblemi dello stesso tipo.
- **Impera**: si risolvono i sottoproblemi ricorsivamente.
- **Combina**: si ricompongono le soluzioni parziali.

Testo su due colonne

Il testo che deve scorrere da una colonna all'altra usa il `columns` di Typst, che bilancia automaticamente le due colonne.

È la soluzione giusta per un paragrafo lungo o per un elenco che non entra in una colonna sola.

Per blocchi indipendenti — un testo e uno schema, del codice e il suo output — conviene invece `side-by-side`, che li affianca senza farli scorrere.

Blocchi affiancati

DEFINIZIONE

Un algoritmo di ordinamento è *stabile* se preserva l'ordine relativo di elementi con chiave uguale.

ATTENZIONE

La stabilità non è gratuita: quicksort in place non è stabile, merge sort lo è al prezzo di memoria ausiliaria.

Definizione, teorema, esempio

TEOREMA

Ogni algoritmo di ordinamento basato su confronti esegue $\Omega(n \log n)$ confronti nel caso peggiore.

DIMOSTRAZIONE

L'albero di decisione ha $n!$ foglie, quindi altezza almeno $\log_2(n!) = \Theta(n \log n)$.

Formule

Il costo di merge sort si ricava dalla ricorrenza

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n), \quad T(1) = \Theta(1),$$

che per il teorema dell'esperto [\[1\]](#) vale $T(n) = \Theta(n \log n)$. In generale, per confronti, $\sum_{i=1}^n \log_2 i = \log_2(n!) = \Theta(n \log n)$.



2 Implementazione

Dal pseudocodice al codice eseguibile.

Frame di codice

MERGE_SORT.PY

```
1 def merge_sort(a):  
2     if len(a) <= 1:  
3         return a  
4     m = len(a) // 2  
5     left, right = merge_sort(a[:m]), merge_sort(a[m:])  
6     return merge(left, right)
```



Due varianti del frame

FONDO PIENO

```
for i in range(n):  
    print(i)
```

CON BORDO

```
for i in range(n):  
    print(i)
```

Codice e output affiancati

SESSIONE

```
1 a = [5, 2, 9, 1, 5, 6]
2 print(merge_sort(a))
3 print(sorted(a) == merge_sort(a))
```

RISULTATO

```
[1, 2, 5, 5, 6, 9]
True
```

Confronto fra algoritmi

Algoritmo	Caso migliore	Caso medio	Caso peggiore	Stabile
Insertion sort	n	n^2	n^2	sì
Merge sort	$n \log n$	$n \log n$	$n \log n$	sì
Quicksort	$n \log n$	$n \log n$	n^2	no
Heapsort	$n \log n$	$n \log n$	$n \log n$	no

Le complessità sono espresse in Θ sul numero di confronti.



Una slide forzata a tutta larghezza

Questa slide segue `#show: wide-mode(true)`, quindi parte dalla prima colonna anche se il deck è nella misura corporate.

Figura con didascalia

schema dell'albero di ricorsione

Albero di ricorsione di merge sort su un vettore di 8 elementi.



L'ottimizzazione prematura è la radice di ogni male.

DONALD E. KNUTH, 1974



3 Conclusioni

Cosa portarsi a casa.

Riepilogo

- Merge sort ordina in $\Theta(n \log n)$ confronti, sempre.
- Il prezzo è $\Theta(n)$ di memoria ausiliaria.
- La **stabilità** è una proprietà dell'algoritmo, non dei dati.
- Per l'analisi completa si vedano [1] e [2]; su quicksort [3].

Riferimenti

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [2] D. E. Knuth, “Structured Programming with go to Statements,” *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [3] R. Sedgewick, “Implementing Quicksort Programs,” *Communications of the ACM*, vol. 21, no. 10, pp. 847–857, 1978.



Domande?